Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский технологический университет «МИСИС»

Институт компьютерных наук (ИКН)

Кафедра Инфокоммуникационных технологий (ИКТ)

Отчет по Лабораторной работе №1

по дисциплине «Технологии программирования»

на тему «Основы объектно-ориентированного программирования»

Выполнил:
студентка группы БИВТ-25-4-6
Муфазалов Э.М
Проверил:

Москва, 2026

Цель работы

Изучить основные принципы объектно-ориентированного программирования и получить практические навыки применения инкапсуляции, наследования, абстрактных классов и полиморфизма при разработке иерархии игровых объектов на языке Python.

Задание

Разработать иерархию игровых объектов пошаговой стратегии. Базовый класс GameObject должен хранить идентификатор, имя и координаты. Класс Unit расширяет базовый класс и реализует работу со здоровьем. Интерфейсы Attacker и Moveable описывают способности атаковать и перемещаться. Класс Archer должен уметь атаковать и перемещаться, Fort — атаковать, а MobileHouse — перемещаться.

1. Ход выполнения работы

1.1. Организация проекта и проектирование иерархии

Работа выполнена на языке Python. Исходный код размещён в файле lab2/main.py в репозитории programming-technologies. Для описания обязательных методов использованы абстрактный базовый класс ABC и декоратор @abstractmethod из модуля abc.

В основе иерархии находится GameObject. От него наследуются Unit и Building. Класс Archer наследуется от Unit и реализует Attacker и Moveable. Класс Fort наследуется от Building и реализует Attacker. Класс MobileHouse наследуется от Building и реализует Moveable.

Класс / интерфейс	Назначение и связи
GameObject	Базовый класс: идентификатор, имя и координаты; методы-геттеры.
Unit	Наследует GameObject; хранит здоровье, проверяет жизнь и обрабатывает урон.
Attacker	Абстрактный интерфейс с методом attack(unit).
Moveable	Абстрактный интерфейс с методом move(x, y).
Archer	Наследует Unit; реализует атаку и перемещение.
Building	Наследует GameObject; хранит состояние постройки.
Fort	Наследует Building; атакует живую цель, если крепость построена.
MobileHouse	Наследует Building; изменяет собственные координаты.

1.2. Реализация интерфейсов Attacker и Moveable

В Python интерфейсные требования заданы через абстрактные классы. Класс Attacker требует реализацию метода attack(unit), а Moveable — метода move(x, y). Метод pass служит заглушкой в абстрактном методе: конкретное поведение определяется в классах-наследниках.

Благодаря абстрактным методам нельзя создать полноценный экземпляр такого класса без реализации требуемых методов в наследнике. Это помогает поддерживать единый интерфейс взаимодействия с игровыми объектами.



1.3. Реализация базового класса GameObject

GameObject хранит идентификатор, имя и координаты объекта в полях _id, _name, _x и _y. Методы get_id(), get_name(), get_x() и get_y() возвращают соответствующие значения. Подчёркивание в начале имён полей обозначает соглашение Python о внутреннем использовании этих атрибутов.

Конструктор __init__ вызывается при создании объекта и устанавливает его начальное состояние. Наследники используют super().__init__(...) для повторного использования инициализации базового класса.

```python
class GameObject:
    def __init__(self, object_id, name, x, y):
        self._id = object_id
        self._name = name
        self._x = x
        self._y = y

    def get_id(self):
        return self._id

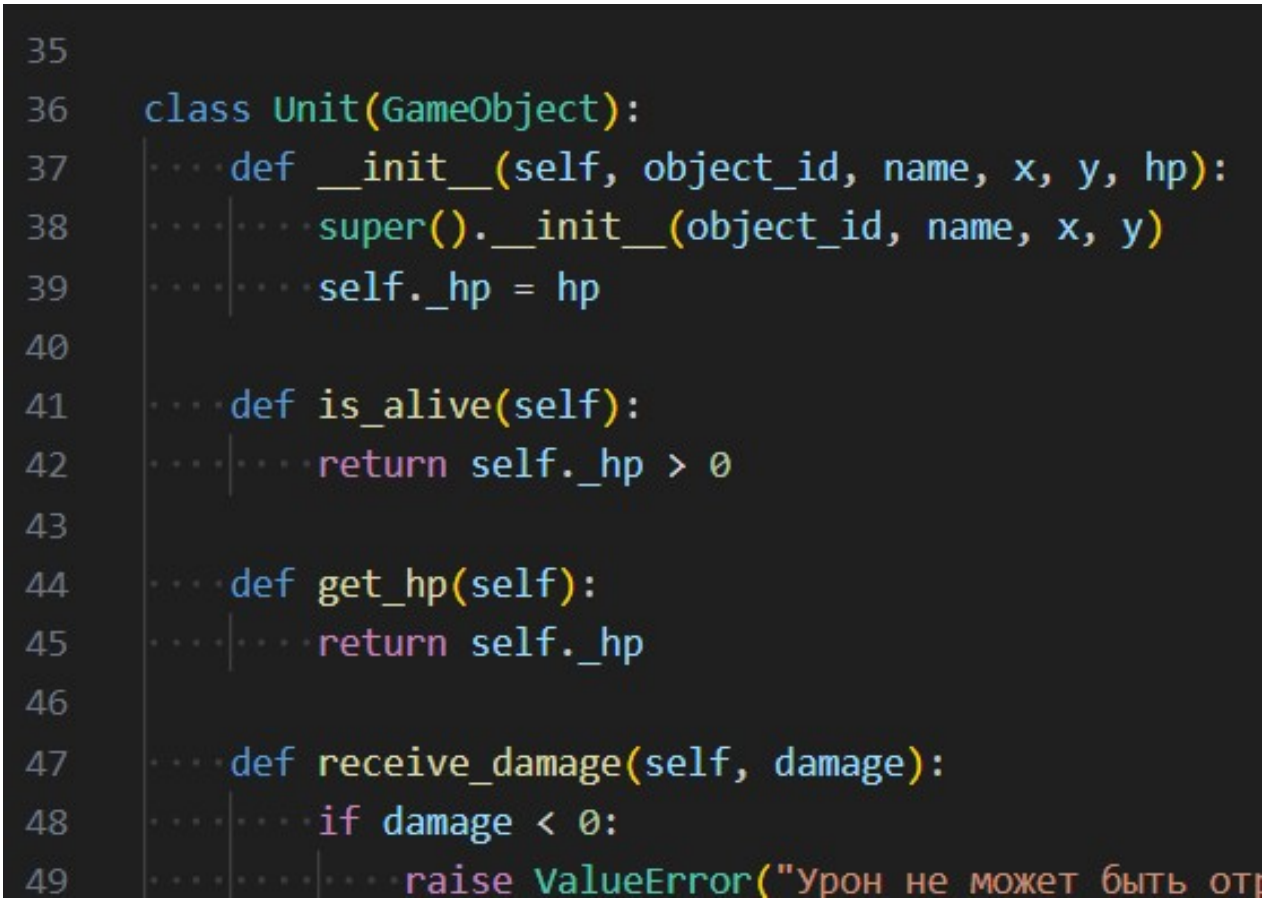
    def get_name(self):
        return self._name

    def get_x(self):
        return self._x
```

1.4. Реализация класса Unit

Unit наследует GameObject и добавляет поле _hp, в котором хранится здоровье юнита. Метод is_alive() возвращает True, если здоровье больше нуля. Метод get_hp() возвращает текущее значение здоровья. Метод receive_damage(damage) уменьшает здоровье на переданный урон.

Проверка отрицательного урона предотвращает некорректные значения. Использование max(0, ...) не позволяет здоровью стать меньше нуля.



```
35
36  class Unit(GameObject):
37      def __init__(self, object_id, name, x, y, hp):
38          super().__init__(object_id, name, x, y)
39          self._hp = hp
40
41      def is_alive(self):
42          return self._hp > 0
43
44      def get_hp(self):
45          return self._hp
46
47      def receive_damage(self, damage):
48          if damage < 0:
49              raise ValueError("Урон не может быть от
```

1.5. Реализация класса Archer

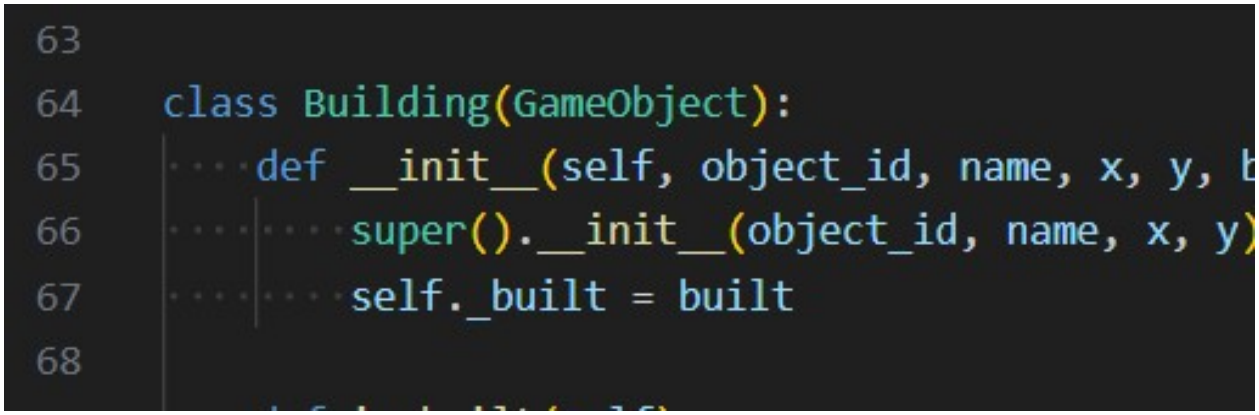
Archer наследуется от Unit и реализует интерфейсы Attacker и Moveable. Метод attack(unit) наносит цели 10 единиц урона, если лучник и цель живы. Метод move(x, y) изменяет координаты лучника.

Таким образом, Archer сочетает свойства юнита с двумя дополнительными возможностями: атакой и перемещением.

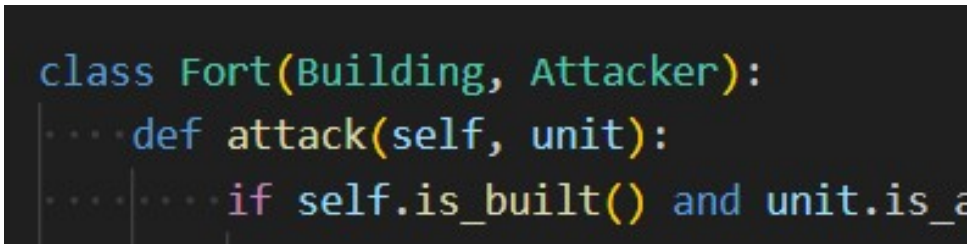


1.6. Реализация построек Building, Fort и MobileHouse

Класс Building наследуется от GameObject и содержит логическое поле _built. Параметр built по умолчанию равен False. Метод is_built() сообщает, построено ли здание.



Fort наследуется от Building и реализует Attacker. Крепость наносит цели 20 единиц урона только в том случае, если она построена и цель жива.



MobileHouse наследуется от Building и реализует Moveable. Метод move(x, y) изменяет координаты дома.

```
78
79  class MobileHouse(Building, Moveable):
80      def move(self, x, y):
81          self._x = x
82          self._y = y
```

1.7. Проверка работы программы

Для проверки в конце main.py был добавлен блок if __name__ == "__main__":. В нём создаются лучник, противник, крепость и мобильный дом. Затем проверяются получение имени и здоровья, состояние жизни юнита, атака, изменение координат и состояние построек.

Программа была запущена командой python lab2/main.py. Получен следующий результат:

Результаты соответствуют логике программы. Противник начинал с 50 единицами здоровья. После атаки лучника его здоровье уменьшилось до 40, а после атаки построенной крепости — до 20. Координаты лучника изменились на (5, 7), а мобильного дома — на (40, 50). Ошибок при запуске не возникло.

```
PS C:\Users\ПК\programming-technologies> python lab2
=== Проверка лучника ===
Имя: Archer
Здоровье: 100
Жив: True
Здоровье врага после атаки: 40
Новые координаты лучника: 5 7
Здоровье врага после атаки крепости: 20
Новые координаты дома: 40 50
```

2. Результаты работы

В ходе работы создана и проверена иерархия игровых объектов. Реализованы базовые классы, наследование, абстрактные интерфейсы и методы, отвечающие за здоровье, атаку и перемещение.

Элемент	Реализованная функциональность	Проверка
GameObject	Идентификатор, имя, координаты и геттеры	Получено имя Archer и координаты
Unit	Здоровье, проверка жизни, получение урона	100 HP у лучника; противник получил урон
Attacker	Абстрактный метод attack(unit)	Реализован в Archer и Fort
Moveable	Абстрактный метод move(x, y)	Реализован в Archer и MobileHouse
Building	Состояние постройки	Проверены Fort и MobileHouse

Исходный код сохранён в Git-репозитории programming-technologies в каталоге lab2/main.py. Изменения зафиксированы коммитом Add OOP lab 2 и успешно отправлены в удалённую ветку main.

Ссылка на репозиторий: https://github.com/Erikqwerty1/programming-technologies

Вывод

В ходе лабораторной работы были изучены и применены основные принципы объектно-ориентированного программирования: инкапсуляция, наследование и полиморфизм. С помощью абстрактных классов Attacker и Moveable заданы общие требования к объектам, способным атаковать и перемещаться. Созданы классы GameObject, Unit, Archer, Building, Fort и MobileHouse. Корректность реализации подтверждена запуском тестового сценария. Исходный код сохранён в Git и опубликован в удалённом репозитории GitHub.